CS405 - Introduction to Computer Graphics Project

1: Transformations, Projection & Curves

Painting: Seker Ahmed Pasa - Kavunlu Naturmort

Sabanci University | Student: Rand MO Khaled-35003| Date: 8 November 2026

1. Introduction and painting selection

I selected Seker Ahmed Pasa's melon still life because its distinctive upper contour provides a natural target for tracing curves. The painting is displayed on a textured plane, with three colored cubes placed in front of it. The project combines transformations, a scene graph, two projection modes and interactive curve editing.

2. Part A - Transformations

Matrices use column-major storage and column vectors. A point is transformed by p_world = M p_local. With translation T, rotation R and scale S, T R S applies scale first, then rotation, then translation; S R T changes the result because matrix multiplication is not commutative. The control panel displays both 4 x 4 matrices and lets the user switch orders. The rest of my code uses T R S, which is the switch's default and what Reset restores, so an object's position is the translation column of its matrix.

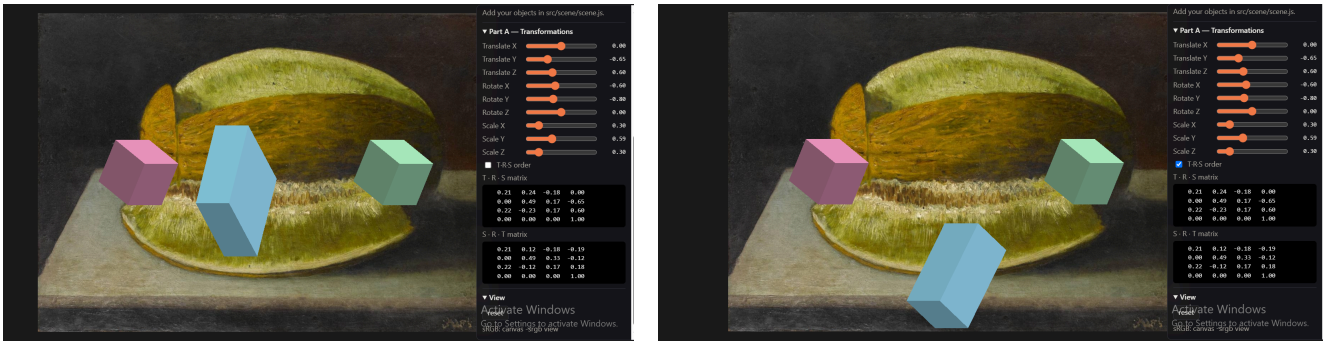


Figure 1. Part A: left, order switch off (S·R·T); right, order switch on (T·R·S). Both matrices are shown in the panel.

The matrix module also contains identity, multiply, translate, rotate X/Y/Z, scale, transpose and rigid inverse operations. The visible differences between T R S and S R T provide a practical check on composition order.

3. Part B - Scene graph and world matrix verification

The scene graph contains five nodes over four hierarchy levels. greenGroup is a transformation-only node (no geometry). Every child inherits the transformations of its ancestors. World matrices are evaluated recursively as W_child = W_parent L_child. The tree is root → blue, and blue has two children: pink and greenGroup, with green under greenGroup. The blue cube is moved by the Part A sliders, so pink and green move with it. pink has the local matrix T(−2.5, 0, 0) and green has T(2.5, 0, 0). greenGroup is the animated node: its local matrix is Ry(θ) with θ growing over time, so the green cube orbits the blue cube. The debug-axes toggle draws local axes.

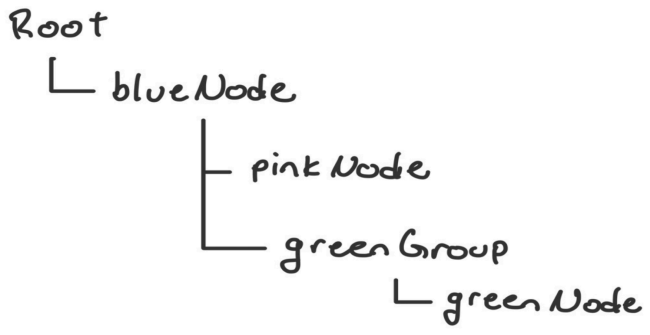


Figure 2. Five-node hierarchy

Hand verification of one leaf

For greenNode, W_green = L_root L_blue L_greenGroup L_green. In the code L_root = I, L_blue = T R S built from the Part A sliders, L_greenGroup = Ry(θ) and L_green = T(2.5, 0, 0). Take the default sliders after Reset (t = (0, 0, 0.6), R = Ry(45°) Rx(30°), scale 0.3) at the start of the animation (θ = 0, so L_greenGroup = I). Then W_green = L_blue T(2.5, 0, 0): the upper-left 3×3 block is R S, and the translation is t + R S (2.5, 0, 0) = (0, 0, 0.6) + 0.75 (cos 45°, 0, −sin 45°) = (0.530, 0, 0.070). Here R S has columns 0.3·(0.707, 0, −0.707), 0.3·(0.354, 0.866, 0.354) and 0.3·(0.612, −0.5, 0.612). For a later θ the same product is used with Ry(θ) between L_blue and T(2.5, 0, 0).

W_green = [0.212 0.106 0.184 0.530 ; 0.000 0.260 −0.150 0.000 ; −0.212 0.106 0.184 0.070 ; 0 0 0 1].

4. Part C - View matrix derivation

Given eye E, target T and reference up U, first construct z = normalize(E - T), x = normalize(U cross z), and y = z cross x. The two cross products produce an orthonormal right/up/backward basis. The view transform projects (p - E) onto these axes, so each translation entry is the negative dot product of a basis axis with E.

V rows: [x_x x_y x_z -dot(x,E)]; [y_x y_y y_z -dot(y,E)]; [z_x z_y z_z -dot(z,E)]; [0 0 0 1]. For E=(0,0,2.8), T=(0,0,0), U=(0,1,0), V = T(0,0,-2.8). If U is parallel to the view direction, choose a nonparallel fallback up vector.

5. Part C - Projection comparison and frustum

Perspective projection makes distant objects appear smaller, while orthographic projection retains size regardless of depth. The two screenshots below show the same painting and three cubes with perspective and orthographic selected. The camera position is approximately (0.02, 0.19, 2.79); near=0.1 and far=8.

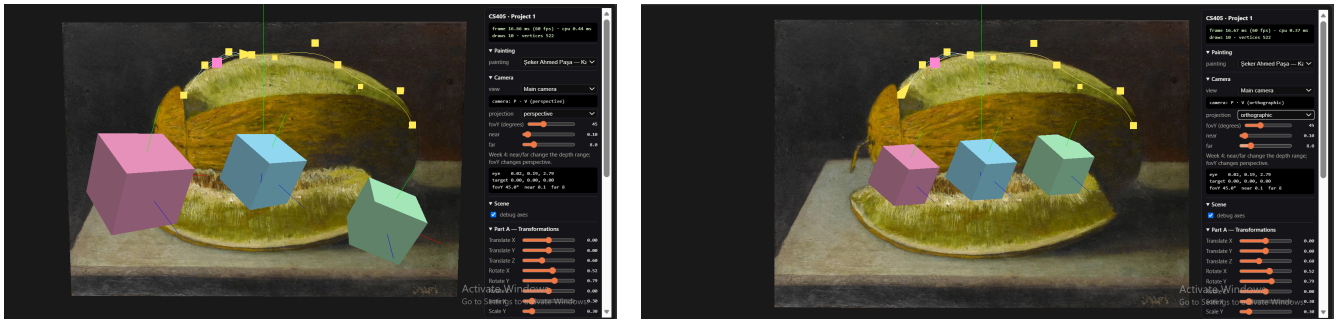


Figure 3. The same scene with perspective (left) and orthographic (right) projection.

For WebGPU depth in [0,1], perspective uses f = 1/tan(fovY/2), horizontal scale f/aspect, vertical scale f, and depth coefficients far/(near-far) and far*near/(near-far). In perspective, depth precision is concentrated near the near plane. Choosing near=0.1 and far=8 gives a ratio of 80:1; a much larger far (for example 100) would extend the range but waste more depth precision on distant geometry. Bringing near closer to zero reduces precision, and objects outside the clipping range disappear.

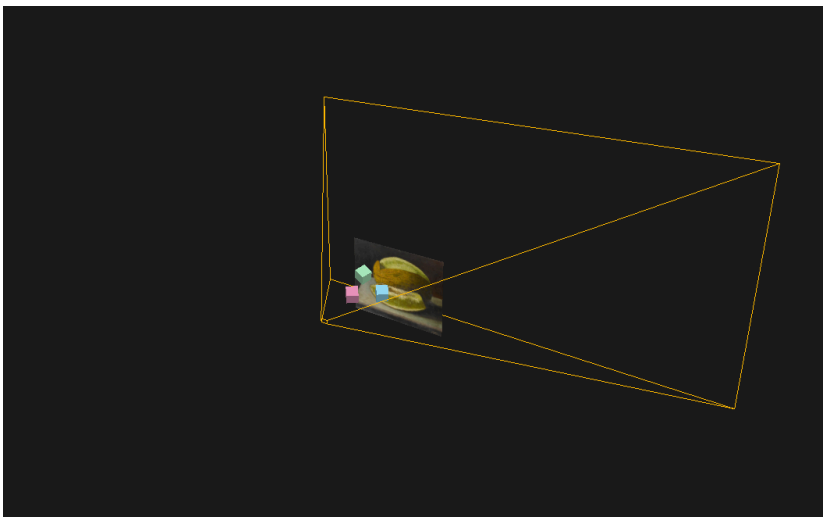


Figure 4. Inspection view: the main camera's frustum .

6. Part D - Curves and mathematical evidence

The cubic Bezier curve is B(t) = b0(t)P0 + b1(t)P1 + b2(t)P2 + b3(t)P3, with t in [0,1]. The Bernstein basis functions are:

b0(t)=(1-t)^3; b1(t)=3(1-t)^2 t; b2(t)=3(1-t)t^2; b3(t)=t^3.

Endpoint proof

At t=0, b0=1 and b1=b2=b3=0, so B(0)=P0. At t=1, b3=1 and b0=b1=b2=0, so B(1)=P3. Thus every segment interpolates its first and last control points.

C1 continuity at a joint

For a first segment ending at J=P3, B1'(1)=3(J-P2). For a second segment starting at Q0=J, B2'(0)=3(Q1-J). Equal derivatives require Q1=2J-P2. The exported data contains three segments with matching endpoints and reflected handles at both joints, satisfying this condition for the stored parameterization.

Traced shape and interactions

The three connected segments trace the upper melon outline. The editor exposes draggable control points, de Casteljau construction, Catmull-Rom display, and a tangent-oriented marker moving along the curve. The exported coordinates are normalized to the painting. Example at the first joint (0.4721, 0.1200): 2·(0.4721, 0.1200) − (0.4163, 0.1100) = (0.5279, 0.1300), which is exactly the next segment's second control point.

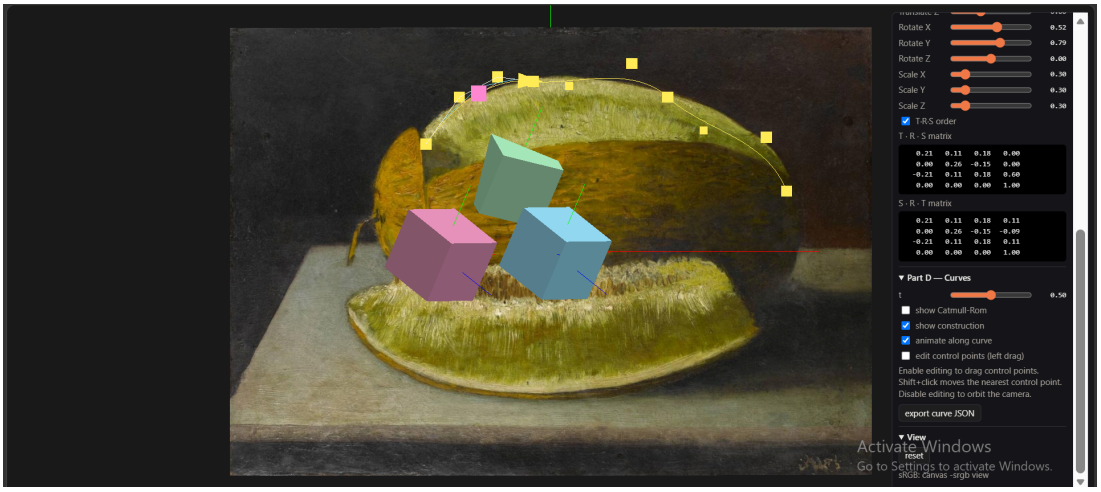


Figure 5. The traced melon outline with control points and the moving marker.

7. Integration, debugging and performance

The application combines Parts A-D in one WebGPU/Vite interface. During development, issues included an initially black canvas caused by buffer/render-pipeline integration, unsupported panel methods (partA.toggle/note/text), camera/reset state inconsistencies, and curve-function signature mismatches. I investigated these through console errors, comparison with course labs, unit tests and visual checks. Debug axes were fixed to draw each node's local axes for root, blue, pink and green; greenGroup's axes are not drawn yet.

approximately 16.66 to 16.67 ms per frame (60 fps), with CPU time around 0.37 to 0.44 ms and 522 vertices submitted over 10 draws in the default view. These are on-screen frame measurements, not GPU execution timings. GPU model: AMD Radeon (TM) Graphics. Browser: Microsoft Edge. For a reproducible benchmark, add resolution and an average over multiple frames.

Appendix A - AI use

Tool used ChatGPT:

Tool	What I asked	What I kept
ChatGPT	As I worked through the project handout, I asked it to explain each section (Parts A–E, the report and the bonus) and which course week or lab would help me solve that part.	The explanations of what each section asks for, and its pointers to the weeks and labs to revisit. I kept no code from these answers.
ChatGPT	After I put my code into the chat, I asked whether it was correct; where it was not, I asked it to fix it.	The corrections it gave for my code, which I applied to the project.
ChatGPT	Part B: I explained that I only had three cubes (and showed a screenshot) and that the scene graph needs a larger tree of at least five nodes. I asked how to reach that.	Its suggestion to add greenGroup, a transformation-only node with no geometry, between the blue and green nodes. This gives the five-node hierarchy.
ChatGPT	Part B: I asked for ideas on how to build an animated arm-and-hand from the scene graph.	Its idea of using the green cube as the "hand" that rotates around the blue cube, so the animation shows the hierarchy.
ChatGPT	I asked it to add all the controls to the panel (Parts A–D, view and reset switches, debug axes, readouts).	The panel controls it provided.
ChatGPT	Asked Chat to format and fix mt report. (gave an initial version with screenshots)	Formatted and fixed report, afterwards I rechecked and fixed chats version

Appendix B - Exported curve (kavunlu-curve.json)

Painting kavunlu, piecewise cubic Bézier with C¹ joints, 3 segments. Control points are in normalised painting coordinates (u, v), origin top-left, v pointing down. Each segment is listed as P0 P1 P2 P3, and the first point of a segment equals the last point of the previous one.

segment 0: (0.3047, 0.2600) (0.3570, 0.1550) (0.4163, 0.1100) (0.4721, 0.1200)

segment 1: (0.4721, 0.1200) (0.5279, 0.1300) (0.6256, 0.0800) (0.6814, 0.1550)

segment 2: (0.6814, 0.1550) (0.7372, 0.2300) (0.8349, 0.2450) (0.8663, 0.3650)